

Sensors in Android

IS2205- Mobile Application Design & Development



Why and What Android Sensors

Add more capabilities to the mobile computer compared to other computing devices such as servers and desktops

The Android platform supports three broad categories of sensors:

- Motion sensors
- Environmental sensors
- Position sensors



Why and What Android Sensors

- Motion sensors - These sensors measure acceleration forces and rotational forces along three axes. i.e.: accelerometers, gravity sensors, gyroscopes, and rotational vector sensors.
- Environmental sensors - These sensors measure various environmental parameters, such as ambient air temperature and pressure, illumination, and humidity. i.e.: barometers, photometers, and thermometers.
- Position sensors - These sensors measure the physical position/orientation of a device. i.e.: orientation sensors and magnetometers.



Some use cases

- A game might track readings from a device's gravity sensor to identify user gestures and motions, such as tilt, shake, rotation, or swing.
- A weather application might use a device's temperature sensor and humidity sensor to calculate and report the dewpoint,
- A travel application might use the geomagnetic field sensor and accelerometer to report a compass bearing.

Question: Find out **another** sensor that may be available in Android devices and describe a use case of it briefly



How can a developer access a sensor?

Android sensor framework - a part of the Android API framework

Sensor framework supported sensor-related tasks.

- Enumerate the sensors available on an Android device.
- Identify an individual sensor's capabilities i.e: maximum range, manufacturer, power requirements, and resolution.
- Acquire raw sensor data and define the rate at which you acquire sensor data.
- **Register** and **unregister** sensor event listeners that monitor sensor changes.



Sensor types

- Hardware based sensors - Physical components built into a device. They derive their data by directly measuring specific physical parameters i.e.: acceleration, geomagnetic field strength.
- Software based sensors - Not physical devices, yet they mimic hardware-based sensors. Software-based sensors derive their data from one or more of the hardware-based sensors. AKA **virtual** sensors or **synthetic** sensors i.e.: linear acceleration sensor, gravity sensor.
- *"Not all mobile devices are made equal !"*
- Most devices have an accelerometer and a magnetometer only a few devices have barometers or thermometers.
- A device can have more than one sensor of a given type. i.e.: 2 gravity sensors, each one having a different range.

Sensor Table

Sensor	Type	Description	Common Uses
TYPE_ACCELEROMETER	Hardware	Measures the acceleration force in m/s^2 that is applied to a device on all three physical axes (x, y, and z), including the force of gravity.	Motion detection
TYPE_LINEAR_ACCELERATION	Software or Hardware	Measures the acceleration force in m/s^2 that is applied to a device on all three physical axes (x, y, and z), excluding the force of gravity.	Monitoring acceleration along a single axis.
TYPE_GRAVITY	Software or Hardware	Measures the force of gravity in m/s^2 that is applied to a device on all three physical axes (x, y, z).	Motion detection (shake, tilt, etc.).
TYPE_AMBIENT_TEMPERATURE	Hardware	Measures the ambient room temperature in degrees Celsius ($^{\circ}\text{C}$). See note below.	Monitoring air temperatures.
TYPE_TEMPERATURE	Hardware	Measures the temperature of the device in degrees Celsius ($^{\circ}\text{C}$). This sensor implementation varies across devices. Was replaced with the TYPE_AMBIENT_TEMPERATURE sensor in API Level 14	Monitoring temperatures.

Sensor Table...



Sensor	Type	Description	Common Uses
TYPE_GYROSCOPE	Hardware	Measures a device's rate of rotation in rad/s around each of the three physical axes (x, y, and z).	Rotation detection (spin, turn, etc.).
TYPE_LIGHT	Hardware	Measures the ambient light level (illumination) in lx (lumens per sq.meter).	Controlling screen brightness.
TYPE_MAGNETIC_FIELD	Hardware	Measures the ambient geomagnetic field for all three physical axes (x, y, z) in μT (microteslas for magnetic flux).	Creating a compass.
TYPE_ORIENTATION	Software	This constant was deprecated in API level 15. use SensorManager.getOrientation() instead.	Determining device position.

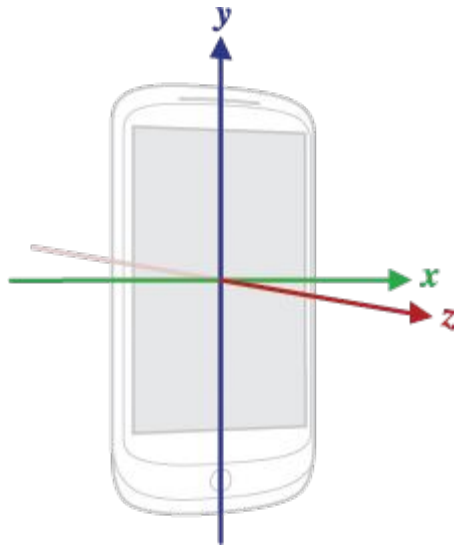
Sensor Table...



Sensor	Type	Description	Common Uses
<u>TYPE_PRESSURE</u>	Hardware	Measures the ambient air pressure in hPa(hectopascal) or mbar(millibar)	Monitoring air pressure changes.
<u>TYPE_PROXIMITY</u>	Hardware	Measures the proximity of an object in cm relative to the view screen of a device. This sensor is typically used to determine whether a handset is being held up to a person's ear.	Phone position during a call.
<u>TYPE_RELATIVE_HUMIDITY</u>	Hardware	Measures the relative ambient humidity in percent (%).	Monitoring dewpoint absolute, and relative humidity.
<u>TYPE_ROTATION_VECTOR</u>	Software or Hardware	Measures the orientation of a device by providing the three elements of the device's rotation vector.	Motion detection and rotation detection.

Sensor Coordinate System

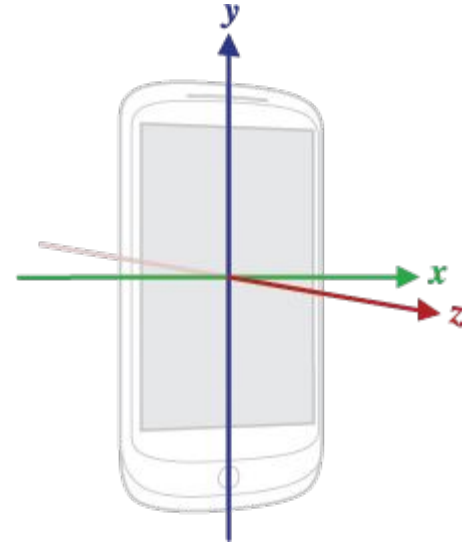
- In general, the sensor framework uses a standard 3-axis coordinate system
- This coordinate system is defined relative to the device's screen when the device is held in its default orientation (see the figure).
- When a device is held in its default orientation(as the figure), the X axis is horizontal and points to the right, the Y axis is vertical and points up, and the Z axis points toward the outside of the screen face.
- In this system, coordinates behind the screen have negative Z values.



Sensor Coordinate System

Questions: When the accelerometer sensor is checked in the given orientation of the device the Y value is approximately +9.8. What will happen to the value if the device starts to free fall from the same position and orientation? Explain the numbers.

For the above orientation if the device is quickly moved to the left side along which axis the value will be changed and what will be the sign of the reading?



Listing and Acquiring sensor object(s)

- Listing all sensors:

```
private lateinit var sensorManager: SensorManager
...
sensorManager = getSystemService(Context.SENSOR_SERVICE) as SensorManager
val deviceSensors: List<Sensor> = sensorManager.getSensorList(Sensor.TYPE_ALL)
```

- SensorManager is the reference point for the developers to acquire a sensor

```
val sensorManager = getSystemService(Context.SENSOR_SERVICE) as SensorManager
val sensor: Sensor? = sensorManager.getDefaultSensor(Sensor.TYPE_GRAVITY)
```

Acquiring a sensor object...Listener

- Example of a sensor class

```
class SensorActivity : Activity(), SensorEventListener {  
  
    private lateinit var sensorManager: SensorManager  
    private var pressure: Sensor? = null  
  
    public override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.main)  
  
        // Get an instance of the sensor service, and use that to get an instance of  
        // a particular sensor.  
        sensorManager = getSystemService(Context.SENSOR_SERVICE) as SensorManager  
        pressure = sensorManager.getDefaultSensor(Sensor.TYPE_PRESSURE)  
    }  
}
```



Acquiring a sensor object...

- Callbacks for Event listening

```
override fun onAccuracyChanged(sensor: Sensor, accuracy: Int) {  
    // Do something here if sensor accuracy changes.  
}  
  
override fun onSensorChanged(event: SensorEvent) {  
    val millibarsOfPressure = event.values[0]  
    // Do something with this sensor data.  
}
```

Acquiring a sensor object...

- Acquiring and Releasing

```
override fun onResume() {  
    // Register a listener for the sensor.  
    super.onResume()  
    sensorManager.registerListener(this, pressure, SensorManager.SENSOR_DELAY_NORMAL)  
}  
  
override fun onPause() {  
    // Be sure to unregister the sensor when the activity pauses.  
    super.onPause()  
    sensorManager.unregisterListener(this)  
}
```

To Read: Important classes on sensors...



SensorManager - You can use this class to create an instance of the sensor service. This class provides various methods for accessing and listing sensors, registering and unregistering sensor event listeners, and acquiring orientation information. This class also provides several sensor constants that are used to report sensor accuracy, set data acquisition rates, and calibrate sensors.

Sensor - You can use this class to create an instance of a specific sensor. This class provides various methods that let you determine a sensor's capabilities.

SensorEvent - The system uses this class to create a sensor event object, which provides information about a sensor event. A sensor event object includes the following information: the raw sensor data, the type of sensor that generated the event, the accuracy of the data, and the timestamp for the event.

SensorEventListener - You can use this interface to create two callback methods that receive notifications (sensor events) when sensor values change or when sensor accuracy changes. May need to change based on the application (i.e.: a mobile game, a drawing application)



Filtering by device capabilities

- If you add this element and descriptor to your application's manifest, users will see your application on Google Play only if their device has an accelerometer.
- You may also set required to false if the app can function even without the feature

```
<uses-feature android:name="android.hardware.sensor.accelerometer"  
             android:required="true" />
```



Best practices : Accessing Sensors

Gather sensor data in the foreground

- On devices running Android 9 (API level 28) or higher, apps running in the background have the restrictions of reporting sensor events

Don't block the `onSensorChanged()` method

- As sensor data can change at a high rate, and the system may call the `onSensorChanged(SensorEvent)` method quite often.
- Do as little as possible within the `onSensorChanged(SensorEvent)` method.
- If required do data filtering or reduction of sensor data flow outside of the `onSensorChanged(SensorEvent)` method.



Best practices : Using APIs

Avoid using deprecated methods or sensor types

- Several methods and constants have been deprecated.
- The TYPE_ORIENTATION sensor type has been deprecated.
- To get orientation data use the **getOrientation()** method instead.
- The TYPE_TEMPERATURE sensor type has been deprecated.
- Use the TYPE_AMBIENT_TEMPERATURE sensor type instead on devices that are running Android 4.0+.

Best practices : Existence and Processing

Verify sensors before you use them

- Always verify that a sensor exists on a device before you attempt to acquire data from it. Don't assume that a sensor exists simply because it's a frequently-used sensor.
- Device manufacturers are not required to provide any particular sensors in their devices.

Choose sensor delays carefully

- When you register a sensor with the `registerListener()` method, be sure you choose a delivery rate that is suitable for your application or use-case.
- Sensors can provide data at very high rates. Allowing the system to send extra data that you don't need wastes system resources and uses battery power.

Best practices : Security

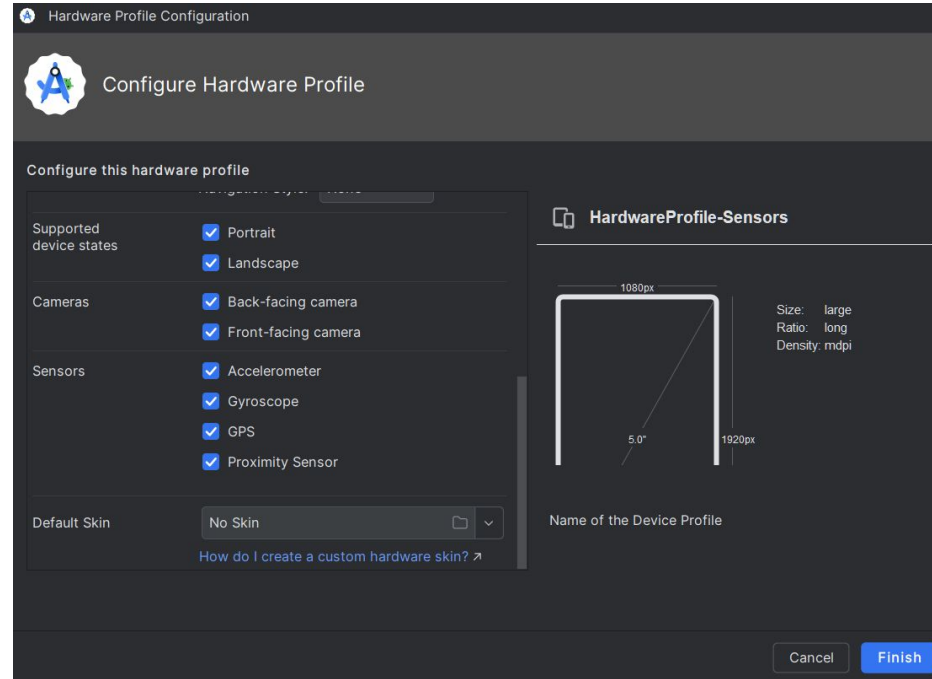
Some sensors may need special permissions

- Due to the privacy concerns some Android sensors may need to get additional permissions from the user.
- BODY_SENSORS_BACKGROUND permission in addition to the existing BODY_SENSORS permission

Best practices : Emulating

Accessing the Virtual Sensors Control Panel

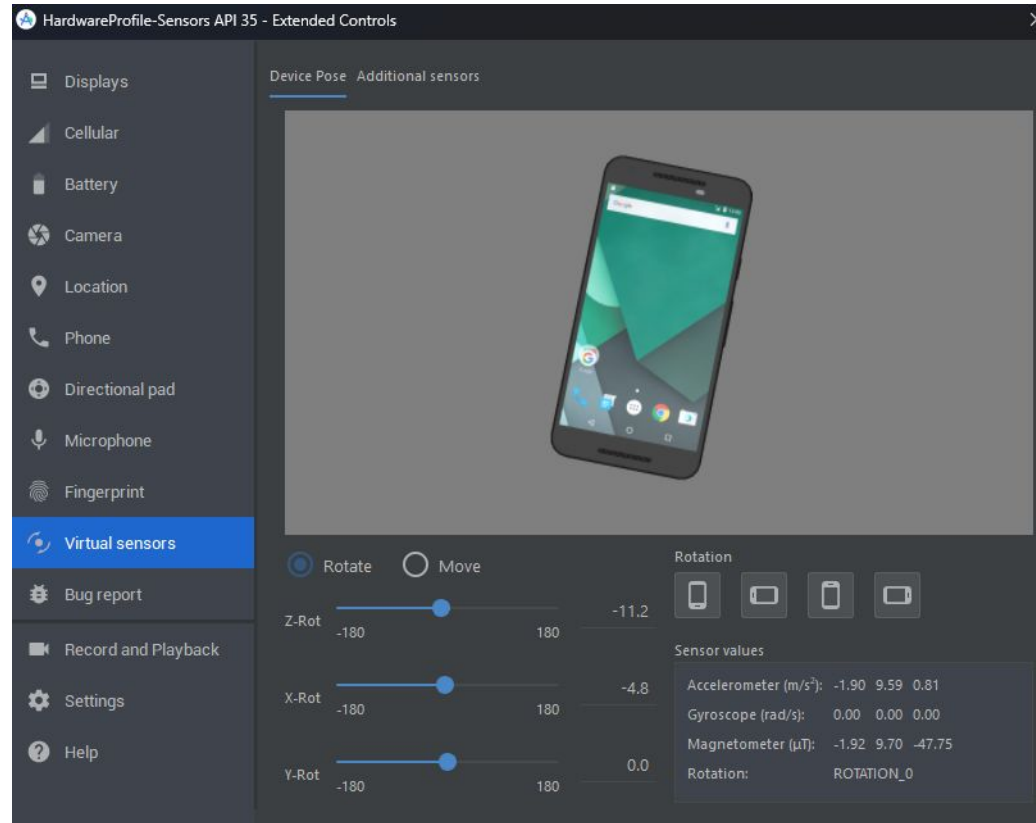
- Launch your emulator inside Android Studio.
- On the emulator's toolbar strip, click the Three Dots (...) icon to open the Extended Controls window.
- In the left-hand menu of the window that pops up, select Virtual Sensors.



Best practices : Emulating...

Mimicking Device Rotation and Tilting

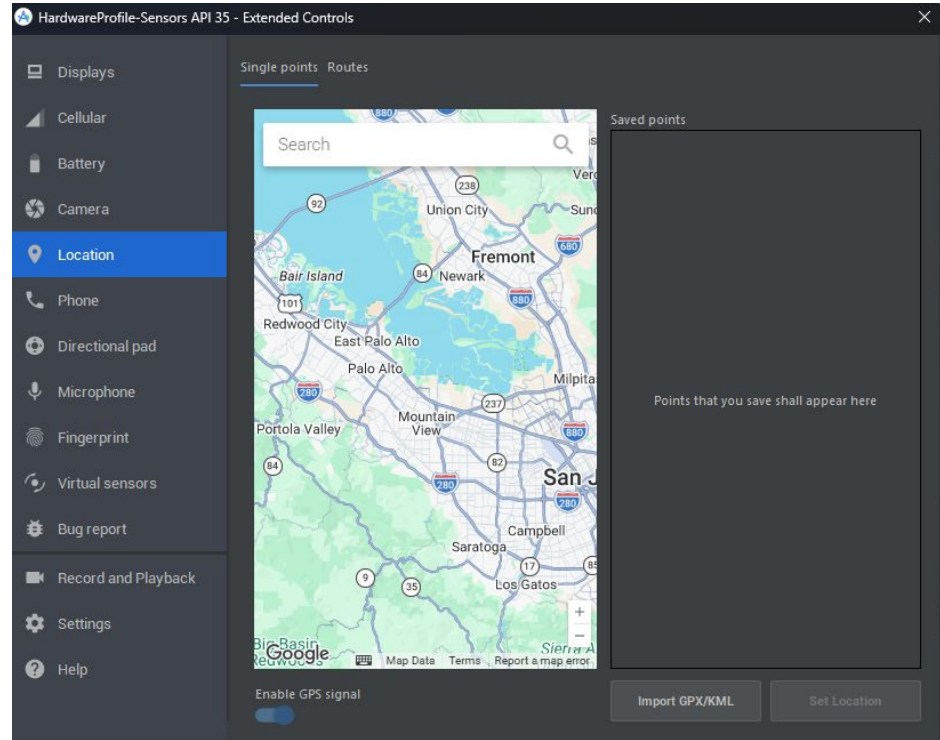
- Under the Device Pose tab, you can simulate moving the phone through 3D space.
- Interactive Preview: Click and drag the 3D phone model in the center of the panel to rotate, spin, or tilt it freely.



Best practices : Emulating...

Mimicking GPS Movement

- Map is interactive and allows marking places by dropping pins.
- Clicking to “teleport” the emulator to those GPS coordinates.
- Allows the routes to be recorded or imported as .gpx or .kml files to simulate walking or driving along a real path.



Question : VLE

Optional Exercise : Use API demonstration code and experiment how Rotational Vector Sensor works:

https://android.googlesource.com/platform/development/+/_master/samples/ApiDemos/src/com/example/android/apis/os/RotationVectorDemo.java

End of the Sensors Section

